Model Evaluation & Selection

A worked-through study reference — train/cv/test splitting, regression, neural networks, and classification, explained from scratch and annotated line by line.

Built from a one-on-one walkthrough of C2W3 Lab 01 · Andrew Ng Machine Learning Specialization, Course 2 (Advanced Learning Algorithms)

Prepared for Nezam · Computer & Systems Engineering, Ain Shams University

1. Splitting the Data & Avoiding Leakage

Why split into train / cv / test at all

A model can always fit the exact data it was trained on well — that alone proves nothing about whether it will work on new data. Splitting the dataset gives you three separate jobs:

- **Training set** — the model actually learns its weights from this.
- **Cross-validation (CV) set** — used to compare different model configurations (polynomial degree, network size, etc.) and pick a winner.
- **Test set** — touched exactly once, at the very end, to report an honest generalization error.

Why the same scaler is reused, not refit per set

A z-score is *not* a general normalization — it's a translation of a raw value into the exact coordinate system the model was trained on: $z = (x - \mu) / \sigma$. The model's weights only make sense relative to the μ and σ computed from training data. If CV or test used their own freshly computed μ/σ, the same raw x would map to a different z than what the model actually learned — producing meaningless predictions.

PYTHON

```
scaler = StandardScaler()
# LEARN mean/std from train AND apply it in one step
X_train_scaled = scaler.fit_transform(x_train)

# Reuse the SAME learned mean/std -- do NOT call fit again
X_cv_scaled = scaler.transform(x_cv)
X_test_scaled = scaler.transform(x_test)
```

Why you must split BEFORE scaling

If you scale the full dataset before splitting, the mean/std used to build every z-score (including training's) is quietly influenced by cv/test values. This is **data leakage**: the measuring ruler itself was calibrated using information the model isn't supposed to have yet, which biases every measurement made with it — even ones made before the leak.

PYTHON

```
# CORRECT order
x_train, x_, y_train, y_ = train_test_split(x, y, test_size=0.40, random_state=1)
x_cv, x_test, y_cv, y_test = train_test_split(x_, y_, test_size=0.50, random_state=1)
# split FIRST

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(x_train) # fit ONLY on train
X_cv_scaled = scaler.transform(x_cv) # apply, never refit
```

Rows vs. columns in the split

The split happens at the **row** level, not per column. Each row is one complete example (its x, its engineered features like x^2, and its true y all belong together). You can't take x from one set of rows and x^2 from another

— that would create examples that don't correspond to anything real. Whichever rows are chosen for train/cv/test, every column for those rows travels together.

Takeaway: z-scores only mean something relative to the mean/std that produced them. Always split rows first, fit the scaler on training rows only, then reuse that exact scaler (never a new one) on cv and test.

2. Regression: Pipeline, Polynomial Features & Model Selection

The 3-step sklearn pattern

Almost every sklearn tool (scaler, model, feature transformer) follows the same lifecycle:

PYTHON

```python
tool = SomeClass() # CREATE -- empty, untrained tool
tool.fit(data) # LEARN -- study data, compute internal numbers
tool.transform(data) # APPLY -- use what it learned
# some tools fuse steps 2+3:
tool.fit_transform(data) # learn AND apply in one call
```

Applying it: scaler and model, annotated

PYTHON

```python
scaler_linear = StandardScaler() # CREATE: knows nothing yet
X_train_scaled = scaler_linear.fit_transform(x_train)
# LEARNS mean/std of x_train, then scales x_train using them

linear_model = LinearRegression() # CREATE: no weights yet
linear_model.fit(X_train_scaled, y_train)
# SOLVES for best-fit w, b directly via the normal equation
# (algebra, not iterative -- no epochs/learning_rate needed)

yhat = linear_model.predict(X_train_scaled) # APPLY: get predictions
mse = mean_squared_error(y_train, yhat) / 2 # compare yhat to true y
```

Why polynomial features

Plain linear regression can only draw a straight line -- it cannot bend to follow a curve no matter how w and b are tuned. The fix is to feed it engineered inputs: adding x^2 as a second column lets it learn $y = w_1x + w_2x^2 + b$. This is still “linear” in the weights, but the resulting curve bends against x.

PYTHON

```python
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_mapped = poly.fit_transform(x_train)
# turns one column [x] into two columns [x, x^2]
# -- no learning here, just reshaping the input

# StandardScaler works PER COLUMN: separate mean/std for x, and for x^2,
# because their raw ranges are wildly different (x ~ thousands, x^2 ~ millions)
scaler_poly = StandardScaler()
X_train_mapped_scaled = scaler_poly.fit_transform(X_train_mapped)
```

The degree-search loop and why CV (not train) decides

Trying degrees 1 through 10 answers a question you can't know in advance: how much bend does this data actually need? Too little underfits (high error everywhere). Too much overfits — training error keeps dropping because the curve is contorting to match training noise, while CV error creeps back up because

that noise doesn't generalize.

PYTHON

```python
for degree in range(1, 11):
    poly = PolynomialFeatures(degree, include_bias=False)
    X_train_mapped = poly.fit_transform(x_train)
    polys.append(poly) # store -- degree 4's poly differs from degree 7's

    scaler_poly = StandardScaler()
    X_train_mapped_scaled = scaler_poly.fit_transform(X_train_mapped)
    scalers.append(scaler_poly) # store the matching scaler too

    model = LinearRegression()
    model.fit(X_train_mapped_scaled, y_train)
    models.append(model) # store the matching model

    train_mses.append(mean_squared_error(y_train, model.predict(X_train_mapped_scaled)) / 2)
    X_cv_mapped_scaled = scaler_poly.transform(poly.transform(x_cv))
    cv_mses.append(mean_squared_error(y_cv, model.predict(X_cv_mapped_scaled)) / 2)

# CV error picks the winner -- train error always looks better with more degrees
degree = np.argmin(cv_mses) + 1 # +1: list index 0 -> degree 1

# Reuse the EXACT stored poly/scaler/model for that winning degree on test -- once
X_test_mapped = polys[degree-1].transform(x_test)
X_test_mapped_scaled = scalers[degree-1].transform(X_test_mapped)
yhat = models[degree-1].predict(X_test_mapped_scaled)
```

What the degree comparison looked like

Degree	Train MSE	CV MSE	Verdict
1	406.19	551.78	Underfits -- straight line can't follow the curve
2	49.11	87.70	Sharp drop in both -- much better fit
4	47.15	79.43	Lowest CV MSE -- selected model
10	< 4	> 120	Overfits -- train keeps dropping, CV rises back up

Takeaway: Polynomial features let a linear model fake curvature. Model selection always reads the **CV** error, never the training error, because training error keeps improving with more flexibility even after the model stops generalizing.

3. Neural Networks: Architecture Comparison

Why no PolynomialFeatures here

Neural networks with hidden layers and non-linear activations (e.g. ReLU) can learn their own curved decision boundary directly from raw inputs — the layers do the “bending” that had to be hand-engineered for plain linear regression. That's why degree=1 (i.e. no added features) is used going into the NN section.

.fit() here is iterative, not algebraic

Plain LinearRegression().fit() solves for weights directly with one algebraic formula — no initialization, no epochs. A neural network has no such closed-form solution once hidden layers are involved, so it needs the same iterative gradient-descent idea from Course 1: start from initial weights, adjust a little each pass, repeat many times.

PYTHON

```
nn_models = utils.build_models() # returns 3 architectures: small, medium, larger

for model in nn_models:
    model.compile(
        loss='mse', # HOW to measure wrongness
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.1), # HOW to search for better weights
    )

    model.fit(
        X_train_mapped_scaled, y_train,
        epochs=300, # run 300 iterative gradient-descent passes over the data
        verbose=0
    )

    yhat = model.predict(X_train_mapped_scaled) # runs the FULL forward pass, all layers
    nn_train_mses.append(mean_squared_error(y_train, yhat) / 2)
    yhat = model.predict(X_cv_mapped_scaled)
    nn_cv_mses.append(mean_squared_error(y_cv, yhat) / 2)
```

Where error is actually calculated

Error is **never** computed per layer. Each layer just passes transformed numbers forward to the next one. Only after the final output layer produces y-hat (the prediction) does the model compare it to the true y — that single difference is the error, which backpropagation then uses to nudge every layer's weights.

1. Input x enters the network.
2. Each hidden layer transforms its input and passes the result forward.
3. The output layer produces one final prediction, y-hat.
4. **Only now** is error computed: compare y-hat to the true y.

Model selection: same rule as regression

Model	Train MSE	CV MSE
1 (small)	73.44	113.87
2 (medium)	73.40	112.28
3 (larger) -- selected	44.56	88.51

Takeaway: Neural networks trade the closed-form solution for iterative training (epochs + learning rate), and can learn curvature on their own from raw inputs. Error is measured once, at the output layer, and the winning architecture is still the one with the lowest **CV** error.

4. Classification: Thresholds, Logits & Sigmoid

Why MSE is replaced

y is now a category (0 or 1), not a continuous number, so “how wrong” isn't a distance — it's whether the predicted class matches. The metric becomes the **fraction misclassified**.

PYTHON

```
probabilities = np.array([0.2, 0.6, 0.7, 0.3, 0.8])
predictions = np.where(probabilities >= 0.5, 1, 0) # apply threshold -> class label
fraction_error = np.mean(predictions != ground_truth) # fraction of wrong classifications
```

Logits vs. sigmoid: why the conversion happens AFTER training

The output layer is left **linear** (raw, unsquashed numbers called logits), and from_logits=True tells the loss function to combine sigmoid + log internally in one numerically stable calculation — this avoids explicitly computing near-0/near-1 probabilities during training, which can destabilize gradients. The cost is that .predict() no longer hands back a probability — you apply sigmoid yourself afterward, only when you want to interpret the result.

PYTHON

```
model.compile(
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
)
model.fit(x_bc_train_scaled, y_bc_train, epochs=200, verbose=0)

yhat = model.predict(x_bc_train_scaled) # raw logits, e.g. 2.0 -- NOT a probability
yhat = tf.math.sigmoid(yhat) # NOW squash to 0-1, e.g. sigmoid(2.0) ~ 0.88
yhat = np.where(yhat >= threshold, 1, 0) # NOW threshold into a class label
train_error = np.mean(yhat != y_bc_train)
```

Matching the output activation to compile()

Output layer activation	compile() setting	predict() gives you
'sigmoid'	from_logits=False (default)	Actual probabilities directly, e.g. 0.88
'linear' / none (used here)	from_logits=True	Raw logits, e.g. 2.0 -- sigmoid yourself

Softmax: same idea, multi-class

For more than two classes, softmax replaces sigmoid: raw logits become a full probability distribution across all classes that sums to 1. Same training trick applies (SparseCategoricalCrossentropy(from_logits=True)), and you apply softmax yourself afterward to interpret results.

PYTHON

```
yhat = model.predict(x) # raw logits, one per class
yhat = tf.nn.softmax(yhat) # real probabilities across classes, summing to 1
predictions = np.argmax(yhat, axis=1) # pick the class with the highest probability
```

The real rule: linear models need poly, neural networks don't

It isn't “regression needs polynomial features, classification doesn't.” It's about whether the model itself is linear:

- **Plain LinearRegression() / plain LogisticRegression()** (no hidden layers) — can only draw a straight line or straight boundary. If the data curves, polynomial features must be added manually, for regression *or* classification.
- **Neural network** (hidden layers + non-linear activations) — learns its own curved boundary from raw features during training, for regression *or* classification. No manual polynomial engineering required.

Takeaway: The linear-vs-network distinction, not the regression-vs-classification one, decides whether you need to hand-engineer polynomial features. Everything else — split first, scale with the training set's scaler only, and let CV error (never train error) choose the winning model — applies identically across regression, neural networks, and classification.